15,709,905 members Sign in

CODE PROJECT® For those who code

arm Get Better Price/Performance with Efficient Kubernetes Walk-through webinar on Aug 9 Register



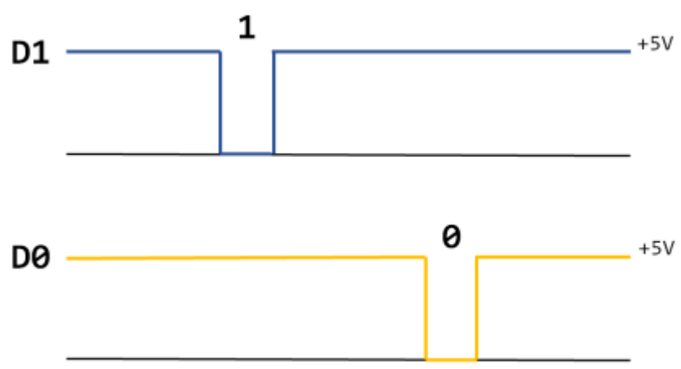
# Wieganduino - Generate Wiegand Codes with Arduino

CPallini
3 Jul 2018 CPOL 3 min read

Rate me: 4.85/5 (6 votes)

Generate 26-bit Wiegand codes with Arduino Nano

Download source - 1.1 KB




***The Wiegand data wires D0 and D1.***

## Introduction

This article shows Wiegand protocol implementation on the Arduino Nano.

## The Wiegand Protocol

The Wiegand protocol, albeit elementary, is still widely used by keyboards (and keycard readers) in access control systems. A brief description follows.

### The Physical Layer

It is a very simple matter:

1. There are `3` physical wires, namely `D0`, `D1` and `GROUND`.
2. When there is no transmission, both `D0` and `D1` are tied to `+5V`.
3. A logical `0` happens when `D0` goes low (`0V`) while `D1` remains high. On the other hand, a logical `1` happens when `D1` goes low while `D0` remains high (see the above picture).
4. Each data bit (either `0` or `1`) must be followed by a pause interval. There are timing requirements for both data pulses and pause intervals (for details and references, please see the Wiegand Wikipedia page).

In my arrangement, the physical layer is represented by the Nano digital pins `2` (`D0`), `3` (`D1`) and `GROUND`. I use it to implement the `26`-bit protocol.

### The 26-bit Protocol

The Wiegand '*message*' is just a `24`-bit numeric code framed by two parity bits:

```
PXXXXXXXXXXXXXXXXXXXXXXXXQ
```

The heading parity bit `P` is added to the leftmost group of `12` bits (MSB) in order to obtain *even* parity. In a similar way, the trailing parity bit `Q` is added to the rightmost group in order to obtain *odd* parity.

For instance, assume the numeric code is `16777215`, i.e., `0xFFFFFF` (all ones):

```
P111111111111111111111111Q
```

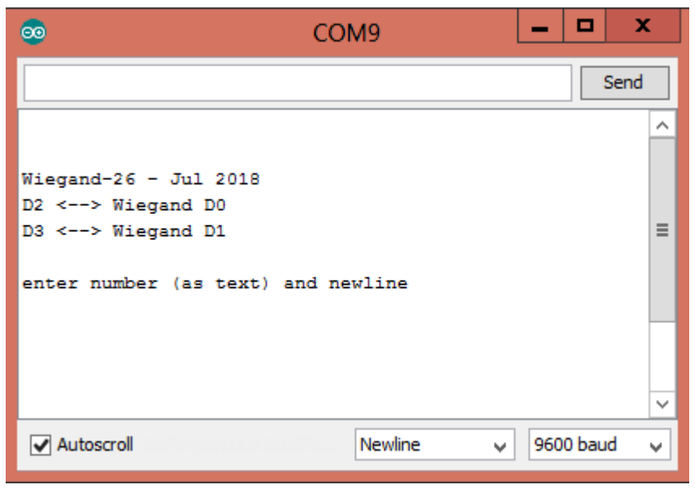
In order to give even parity to the leftmost group of bits, we have to set `P=0`. On the other hand, in order to obtain odd parity in the rightmost one, we have to set `Q=1`.

Hence the final `26`-bit code is:

```
01111111111111111111111111
```

## Using the Sketch

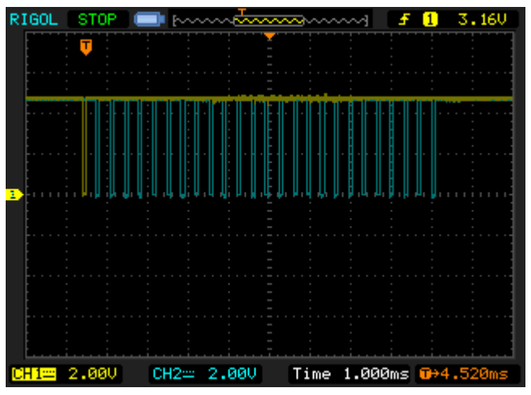
Open the Serial Monitor tool: Arduino waits for a numeric code followed by a newline (see the picture below).



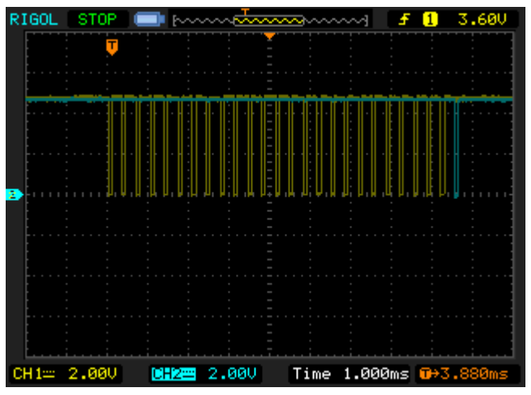

***The Serial Monitor Tool***

On successful reception of the numeric code, the corresponding Wiegand message is produced on digital pins `2, 3`.

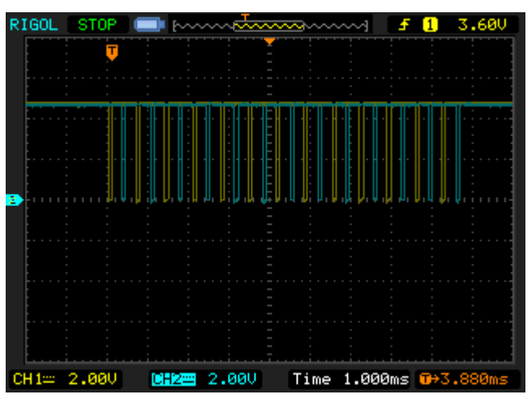
Some patterns captured by my trustworthy RIGOL scope:



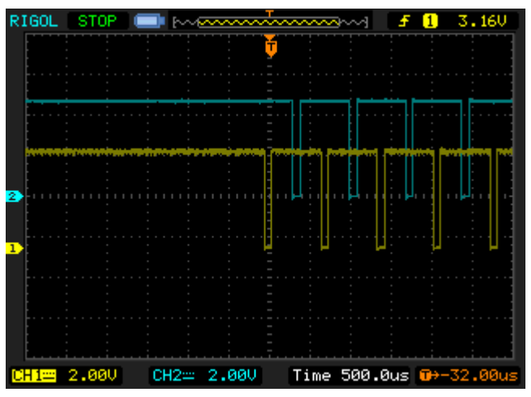

***The 16777215 (0xFFFFFF) code: yellow channel is Wiegand D0, blue channel is D1***




***The 0 code: note the last bit set to 1 in order to obtain trailing odd parity.***



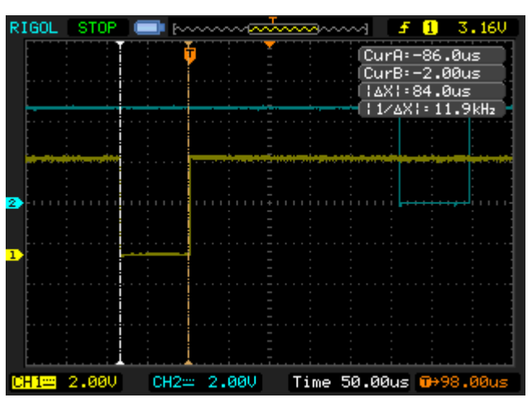

***The 11184810 (0xAAAAAA) code flips all the bits.***



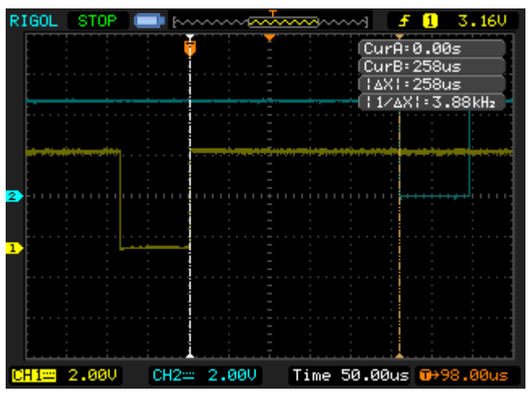

***This is a particular of the 0xAAAAAA pattern, showing the idle-to-data transition of the communication line.***

## Timings

In order to fulfill Wiegand timing constraints, the Arduino `delayMicroseconds` function is used. While results are not brilliant, protocols requirements are met.




***Data pulse timing.***




***Interval pulse timing.***

## The Code

The code is simple and heavily commented. The workhorse functions are reported in the below snippets:

```c
// outputs ONE Wiegand bit
void outwiegbit(unsigned int b)
{
  int sel = b == 0 ? W_D0 : W_D1;
  digitalWrite(sel, 0);
  delayMicroseconds(80);
  digitalWrite(sel, 1);
  delayMicroseconds(240);
}
```

The `outwiegbit` trivially outputs a bit on Wiegand outputs with proper timings:

```c
// outputs a 26 bit Wiegand code
// u32 is actually the 24-bit numeric code
void outwieg26(uint32_t u32)
{
  uint32_t tmp = u32;
  unsigned int p_even = 0;
  unsigned int p_odd = 1;
  // compute parity on trailing group of bits
  for (int n=0; n<12; ++n)
  {
    p_odd ^= (tmp & 1);
    tmp >>= 1;
  }
  // compute parity on heading group of bits
  for (int n=12; n<24; ++n)
  {
    p_even ^= (tmp & 1);
    tmp >>= 1;
  }
  // now output data bits framed by parity ones
  outwiegbit(p_even);
  for (int n=0; n<24; ++n)
  {
    outwiegbit((u32 >> (23-n)) & 1);
  }
  outwiegbit(p_odd);
}
```

The `outwieg26` outputs a 26-bit code. Heading (even) and trailing (odd) parity bits are first computed on the actual 24-bit code.

# Points of Interest

Building a Wiegand code generator on Arduino was really quick.

Another step could be adding a keypad in order to create a real standalone control access device.

# History

- 2nd July, 2018 - First revision

# License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

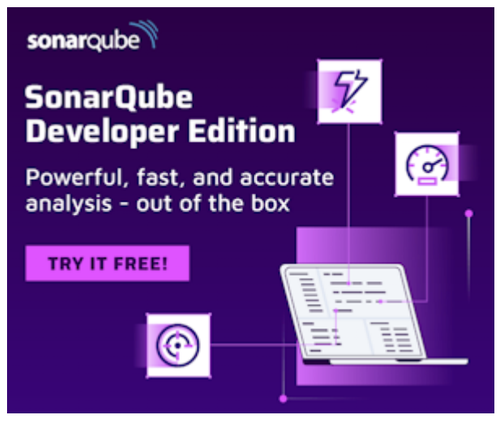
Written By
**CPallini**
Software Developer (Senior) Biotecnica Instruments S.p.A.
Italy

Debugging? Klingons do not debug. Our software does not coddle the weak. Bugs are good for building character in the user.
-- The Klingon programmer

---

**Beelzebub** for his friends [^].

Article
Browse Code
View Stats
Revisions (3)
Comments (14)
Posted 3 Jul 2018
Tagged as
C
Arduino
Stats
32.8K views
1.4K downloads
3 bookmarked



# Comments and Discussions

You must Sign In to use this message board.

Search Comments

Spacing Relaxed Layout Normal Per page 25 Update

First Prev Next

| | | |
|---|---|---|
| Keypad? | Member 14568196 | 7-May-23 10:26 |
| i need to output same card number every time we short analog pin | Member 15391144 | 14-Oct-21 12:49 |
| Re: i need to output same card number every time we short analog pin | CPallini | 14-Oct-21 20:10 |
| Thank!!! | Rodolfo Barros | 10-Oct-21 6:08 |
| Re: Thank!!! | CPallini | 11-Oct-21 2:08 |
| Re: Thank!!! | Rodolfo Barros | 11-Oct-21 4:36 |
| Re: Thank!!! | CPallini | 11-Oct-21 5:25 |
| Convert | Nik0099 | 7-Apr-21 6:52 |
| Thanks | Member 14718086 | 17-Jan-20 3:11 |
| about the timing | gerard k | 8-Sep-19 12:22 |
| Weigand Output | Member 14561380 | 17-Aug-19 6:52 |
| awesome | jow88s | 2-Jan-19 23:59 |
| Interesting article | Mike Hankey | 3-Jul-18 7:14 |
| Re: Interesting article | CPallini | 3-Jul-18 7:32 |

Last Visit: 31-Dec-99 18:00 Last Update: 5-Aug-23 11:50 Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2018 by CPallini
Everything else Copyright © CodeProject, 1999-2023

Web02 2.8:2023-07-24:2